Empirical Analysis of Sparse Autoencoders: Effects of Regularization Methods and Hyperparameters on Parts-Based Representations

Maksymilian Deryło, Lan de Jong

March 22, 2026

reduce margin at the end

Abstract

Frameworks utilizing sparse autoencoders (SAEs) provide an unsupervised approach to learn interpretable features from densely encoded data. Similiar to non-negative matrix factorization (NMF), where non-negativity constraint yields the decomposition into additive parts-based representation that are highly interpretable by humans, SAEs aim to achieve analogous decompositions through sparsity constraints. In this study, we conduct an empirical analysis of different SAE implementations, focusing on comparative behavior of Top-K and Kullback-Leibler (KL) sparsity terms, techniques to reduce the amount of dead-neurons, such as weight initialization, auxiliary loss, and JumpReLU, as well as hyperparameters effect on loss and interpretability.

TODO add a sentence of findings afterwards

.

1 Introduction

TODO Slightly longer description than the one in the abstract, datasets used - MNIST, olivietti maybe imagenet. Look into the actual tests performed

2 Methods

2.1 Datasets

In this study we focus on parts-based representations, thus we test our hypotheses on normalized datasets frequently used for representation learning research: the MNIST handwritten digits dataset[LBBH98], the Olivetti Faces dataset[SH94], and a subset of the ImageNet dataset [DDS+09]. These datasets provide increasing complexity, from simple digit structure to high-dimensional natural images, allowing the implementation of the scientific method designing reproducible experiments across varying data modalities.

Preprocessing All datasets undergo consistent preprocessing to ensure non-negative inputs suitable for parts-based decomposition. For MNIST, images are converted to tensors with pixel values in the range $[0, 1]$ and flattened to vectors of dimension $d = 784$. The Olivetti Faces dataset is similarly normalized to $[0, 1]$ and resized to a consistent resolution. ImageNet samples are center-cropped, resized, and normalized. Following standard practice in SAE training [GDlTT+24a], we subtract the mean activation vector computed over the training set and normalize each input vector to unit norm:

$$x' = \frac{x - \mu}{\|x - \mu\|_2} \tag{1}$$

where μ is the mean input vector. This normalization ensures that reconstruction error metrics are comparable across different input magnitudes and prevents the autoencoder from exploiting scale information rather than learning meaningful sparse features [GDlTT+24a].

2.2 SAE formulations

Architecture A sparse autoencoder (SAE) is a variant of the traditional autoencoder designed to learn efficient and interpretable latent representations by encouraging hidden units to activates sparsely [Ng11]. For an input vector $x \in \mathbb{R}^d$, the encoder and decoder functions are defined by:

$$h = \sigma(W_{\text{enc}}x + b_{\text{enc}}) \tag{2}$$

$$\hat{x} = W_{\text{dec}}h + b_{\text{dec}} \tag{3}$$

with $W_{\text{enc}} \in \mathbb{R}^{n\times d}$, $b_{\text{enc}} \in \mathbb{R}^n$, $W_{\text{dec}} \in \mathbb{R}^{d\times n}$, $b_{\text{enc}} \in \mathbb{R}^d$, the encoder and decoder parameter weights respectively. $h \in \mathbb{R}^n$ is the hidden layer representation and $\hat{x} \in \mathbb{R}^d$ the reconstruction vector. Furthermore, contrary to standard formulations of SAEs we additionally clamp the decoder weights W_{enc} to be non-negative. With that, the learned hidden layer activations, also non negative from the activations functions we use, will use the "building blocks" of decoder weights to reconstruct the image. This can be done, as previously we preprocess the data to be non-negative as well.

The general objective function for the training then becomes:

$$\mathcal{L} = \mathcal{L}_{\text{recon}} + \lambda\mathcal{L}_{\text{sparse}} \tag{4}$$

where $\mathcal{L}_{\text{recon}} = \|x - \hat{x}\|_2^2$ denotes mean squared reconstruction error, $\mathcal{L}_{\text{sparse}}$ is a regularization term promoting sparsity in the activations h, weighed by the coefficient λ.

KL-Divergence with ReLU The KL-divergence formulation enforces sparsity by penalizing deviations from a target average activation level across the training data [Ng11]. For a hidden unit j, let $\hat{\rho}_j = \frac{1}{m}\sum_{i=1}^m h_j^{(i)}$ denote the empirical average activation over m training samples, where $h_j^{(i)}$ is the activation of neuron j for input $x^{(i)}$. The sparsity penalty is defined as the sum of Kullback-Leibler divergences between a target sparsity parameter ρ (typically $\rho \ll 1$) and the empirical activation $\hat{\rho}_j$:

$$\mathcal{L}_{\text{sparse}} = \sum_{j=1}^{n} \text{KL}(\rho\|\hat{\rho}_j) = \sum_{j=1}^{n}\left[\rho\log(\frac{\rho}{\hat{\rho}_j}) + (1-\rho)\log(\frac{1-\rho}{1-\hat{\rho}_j})\right] \tag{5}$$

It is often combined with the rectified linear unit (ReLU) activation function

$$\sigma(z) = \max(0, z) \tag{6}$$

Another proposed approach we test against is to instead combine KL-Divergence with Jump ReLU activation function[RLS+24] defined as:

$$\text{JumpReLU}_\theta(z) = zH(z-\theta) \tag{7}$$

where H is the Heaviside step function. These functions maintain non-negative activations while encouraging most hidden units to remain inactive for any given input [GBB11].

TopK Activation The TopK activation function provides an alternative approach to enforcing sparsity by directly enforcing non-negativity and selecting the K largest pre-activations and zeroing out all others [GDlTT+24a]. For encoder pre-activations $\pi(x) \in \mathbb{R}^n$, the TopK function is defined as:

$$\text{TopK}_K(\pi(x))_j = \begin{cases} \pi_j(x) & \text{if } \pi_j(x) \text{ is among the top } K \text{ values} \\ 0 & \text{otherwise} \end{cases} \tag{8}$$

Unlike KL-divergence or L1 regularization approaches, TopK SAEs do not require an explicit sparsity penalty term in the loss function, as sparsity is enforced directly through the activation function [RLS+24], thus it simply becomes $\mathcal{L} = \mathcal{L}_{\text{recon}}$.

2.3 Dead Neuron Mitigation Techniques

Training sparse autoencoders frequently encounters the challenge of dead neurons—latent features that rarely or never activate during training, resulting in inefficient capacity utilization and reduced representational power [RLS+24, GDlTT+24a]. This phenomenon is particularly pronounced in TopK architectures where the hard sparsity constraint can lead to feature competition, causing some neurons to become permanently inactive. We investigate two complementary mitigation strategies: weight initialization schemes and auxiliary loss mechanisms.

2.3.1 Auxiliary Loss for Feature Resurrection

The auxiliary loss mechanism explicitly penalizes poor reconstruction quality when utilizing currently inactive features, thereby incentivizing the model to resurrect dead neurons [GDlTT+24a]. We define a feature as dead if it has not activated for more than a threshold number of training steps τ_{dead}. Let $\mathcal{D}_t \subseteq \{1, \ldots, n\}$ denote the set of dead feature indices at training step t, determined by tracking steps since last activation for each neuron j:

$$j \in \mathcal{D}_t \iff s_j > \tau_{\text{dead}} \tag{9}$$

where s_j counts consecutive steps without activation (reset to zero when $|h_j| > \epsilon$ for threshold $\epsilon = 10^{-8}$).

For a given input x, the auxiliary reconstruction targets the residual error $e = x - \hat{x}$ from the primary TopK reconstruction. We compute auxiliary activations z_{aux} by selecting the top-K_{aux} features exclusively from dead neurons:

$$z_{\text{aux}} = \text{TopK}_{K_{\text{aux}}}(h_{\text{raw}} \odot \mathbb{1}_{\mathcal{D}_t}) \tag{10}$$

where $h_{\text{raw}} = W_{\text{enc}}x + b_{\text{enc}}$ are pre-activation values and $\mathbb{1}_{\mathcal{D}_t}$ is the indicator vector for dead features. The auxiliary reconstruction is then:

$$\hat{e} = W_{\text{dec}} z_{\text{aux}} + b_{\text{dec}} \tag{11}$$

The auxiliary loss penalizes discrepancy between the actual residual error and the reconstruction from dead features:

$$\mathcal{L}_{\text{aux}} = \frac{1}{d}\|e - \hat{e}\|_2^2 = \frac{1}{d}\|(x - \hat{x}) - W_{\text{dec}} z_{\text{aux}}\|_2^2 \tag{12}$$

Following established practice [GDlTT+24a, LBMP+25], the auxiliary loss is weighted by a small coefficient $\alpha = \frac{1}{32}$ to avoid overwhelming the primary reconstruction objective. The complete loss function becomes:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{recon}} + \alpha \mathcal{L}_{\text{aux}} \tag{13}$$

This formulation creates a direct gradient pathway for dead features: to minimize total loss, the model must ensure that dead features, when activated, produce useful reconstructions of the residual error. We set $K_{\text{aux}} = 2K$ to provide sufficient opportunities for feature resurrection while maintaining computational efficiency.

Dead Feature Tracking During training, we maintain a buffer $s \in \mathbb{N}^n$ tracking steps since activation for each feature. After computing activations h for a batch, we update:

$$s_j \leftarrow \begin{cases} 0 & \text{if } \max_i |h_j^{(i)}| > \epsilon \\ s_j + 1 & \text{otherwise} \end{cases} \tag{14}$$

where the maximum is taken over all samples in the batch. This batch-level tracking ensures that features active on any training example are not prematurely classified as dead.

2.3.2 Weight Initialization Schemes

Tied Weights Initialization We compare no intentional initialization against random Xavier initialization [GB10] with tied weight initialization where $W_{\text{dec}} = W_{\text{enc}}^T$ which is aimed at reducing the amount of dead neurons [GDlTT+24a].

3 Comparative Analysis

3.1 KL-Divergence Sparsity Analysis

In KL-Divergent SAE's, two parameters govern the balance between reconstruction fidelity and feature selectivity. The KL-Divergence weight, denoted as λ, denotes the strength of the sparsity penalty term in the loss function. The sparsity target, denoted as ρ, controls the desired average activation level for each hidden neuron across all training examples.

describe the experiments, how they have been performed, constant params etc. intuitively higher gradient will be affected more by the sparsity loss and not the reconstruction, thus it will result in a worse one - visualize the effect on the actual reconstruction between those.

3.1.1 Hyperparameter Sensitivity

Hypothesis

Experimental setup

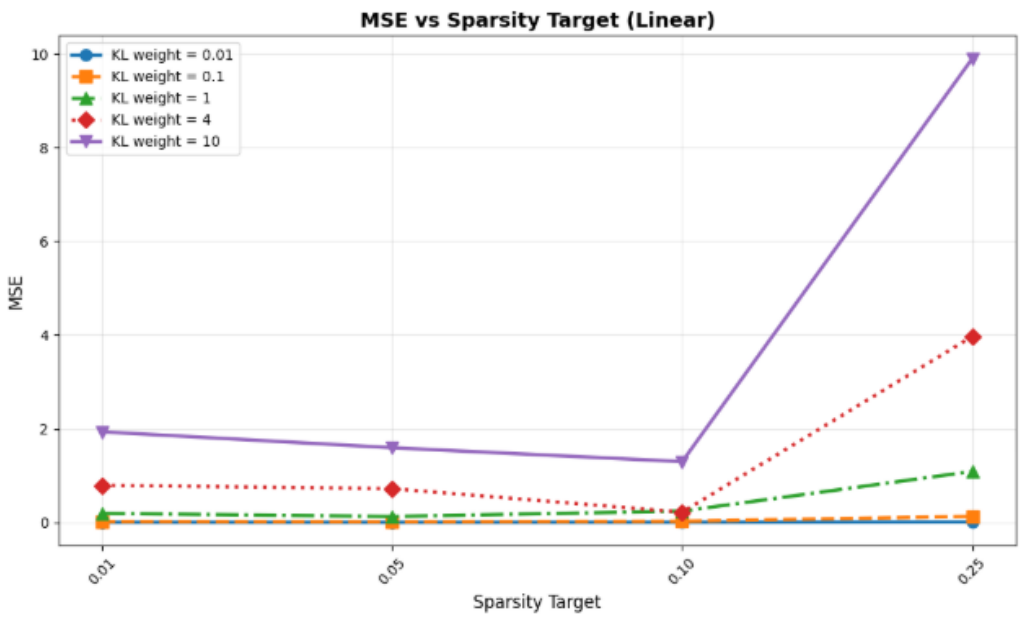

Figure 1: Sparsity Target versus MSE for different KL weights

Results

3.1.2 Activation Function Comparison: ReLU vs JumpReLU

- Hypothesis: JumpReLU reduces feature shrinkage - Experimental setup - Results, Figures

3.2 TopK Sparsity Analysis

The TopK activation function provides direct sparsity control by retaining only the k largest pre-activations while zeroing all others, thereby eliminating the need for explicit L_1 regularization [GDlTT+24b, MF13]. Unlike soft sparsity constraints that introduce activation shrinkage bias, TopK prevents this phenomenon by removing the L_1 penalty entirely [GDlTT+24b]. Although TopK enforces sparsity in the pre-activations, sparsity alone does not guarantee semantic meaningfulness of retained activations [BPY+21]. Apparent strong activations on a single dataset may fail to generalize across domains, creating an "interpretability illusion" where features appear monosemantic but lack true generalizability

[BPY+21]. In this work we evaluate how TopK affects reconstruction fidelity, dead neuron prevalence, and parts-based representation quality, reporting mean squared error (MSE) for reconstruction quality alongside the number of dead latents [GDlTT+24b].

3.3 Tackling Dead Neurons

A persistent challenge in training sparse autoencoders (SAEs) is the emergence of dead neurons—latent features that rarely or never activate during training [GDlTT+24b, RLS+24]. This phenomenon limits effective dictionary utilization and reduces representational capacity, particularly under hard sparsity constraints such as TopK where feature competition causes permanent inactivity [RLS+24]. We investigate two complementary mitigation strategies: tied weight initialization for prevention and auxiliary loss for resurrection [GDlTT+24b].

3.3.1 Experimental Setup

We compare four configurations across MNIST, Olivetti Faces, and LFW datasets: (1) baseline *SAE* without specific initialization [GB10], (2) *SAE_Init* with tied weights and Xavier initialization ($W_{\text{dec}} = W_{\text{enc}}^T$), (3) *SAE_AuxLoss* with auxiliary loss (see Methods), and (4) *Complete SAE* combining both techniques [GDlTT+24b]. All TopK models use $k = 32$ and are trained for 50 epochs (batch size 128, $\eta = 10^{-3}$, Adam optimizer) [KB14]. We evaluate multiple hidden dimensions: $n = 128$ (MNIST), $n \in \{256, 512, 1024\}$ (Olivetti), and $n \in \{512, 1024, 2048\}$ (LFW) to assess scalability. We analyze dead neuron prevalence, activation frequency distributions, and activation strength distributions [GDlTT+24b].

perform experiment with independent xavier encoder decoder

3.3.2 Weight Initialization Results

Figure 2 shows that *SAE_Init* achieves the lowest dead neuron rates across all datasets and dimensions, with 30-47% relative reductions versus baseline [GDlTT+24b]. On MNIST ($n = 128$), tied initialization reduces dead neurons from 70% to 37%, approaching the 39% of unconstrained models [GDlTT+24b]. For Olivetti at $n = 256$, the improvement is from 70% to 46%; for LFW at $n = 512$, from 90% to 75% [GDlTT+24b]. Dead neuron prevalence increases with hidden dimension for all configurations except those using tied initialization, indicating that proper initialization becomes critical at scale [GDlTT+24b].

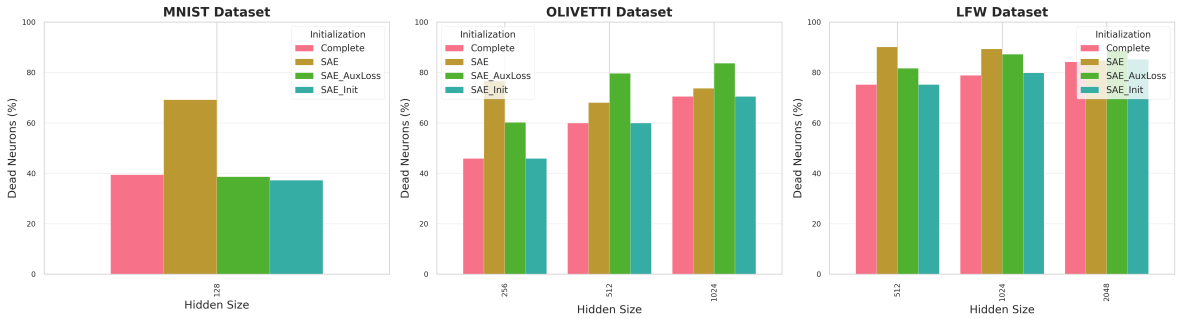

Figure 2: Dead neuron percentage across initialization strategies for three datasets. *SAE_Init* (tied weights) consistently achieves lowest rates with 30-47% relative reductions. *SAE_AuxLoss* provides intermediate improvements. *Complete* (combined techniques) shows modest additional gains. Dead neuron prevalence increases with model capacity under TopK constraints [GDlTT+24b].

Tied initialization promotes balanced feature discovery through symmetric gradient flow, preventing early asymmetric updates that permanently silence neurons [GB10]. This preventive approach is more effective than post-hoc resurrection, offering practical advantages for large-scale training where auxiliary loss computations are expensive [GDlTT+24b].

3.3.3 Auxiliary Loss and Combined Effects

While auxiliary loss alone achieves moderate 10-15% improvements, Figure 3 reveals its critical contribution to feature quality when combined with initialization [GDlTT+24b]. The top row shows activation frequency distributions: baseline SAE exhibits pathological concentration with 100+ (MNIST), 220+ (Olivetti), and 480+ (LFW) features at near-zero frequency, plus isolated high-frequency spikes indicating winner-take-all dynamics [GDlTT+24b]. Complete SAE reduces near-zero features by 40-50% and eliminates high-frequency spikes, achieving balanced load distribution across 0.05-0.20 frequency range [GDlTT+24b].

The bottom row demonstrates substantial activation strength improvements [GDlTT+24b]. Baseline SAE shows weak activations (0.05-0.10), while Complete SAE achieves 3-4× stronger activations (0.15-0.30 depending on dataset) [GDlTT+24b]. This strength increase indicates more confident, interpretable features rather than noisy activations capturing residual variance [OF96]. Stronger activations correspond to more salient decoder weight patterns, directly enhancing interpretability of the learned parts-based representation [OF96, LS99].

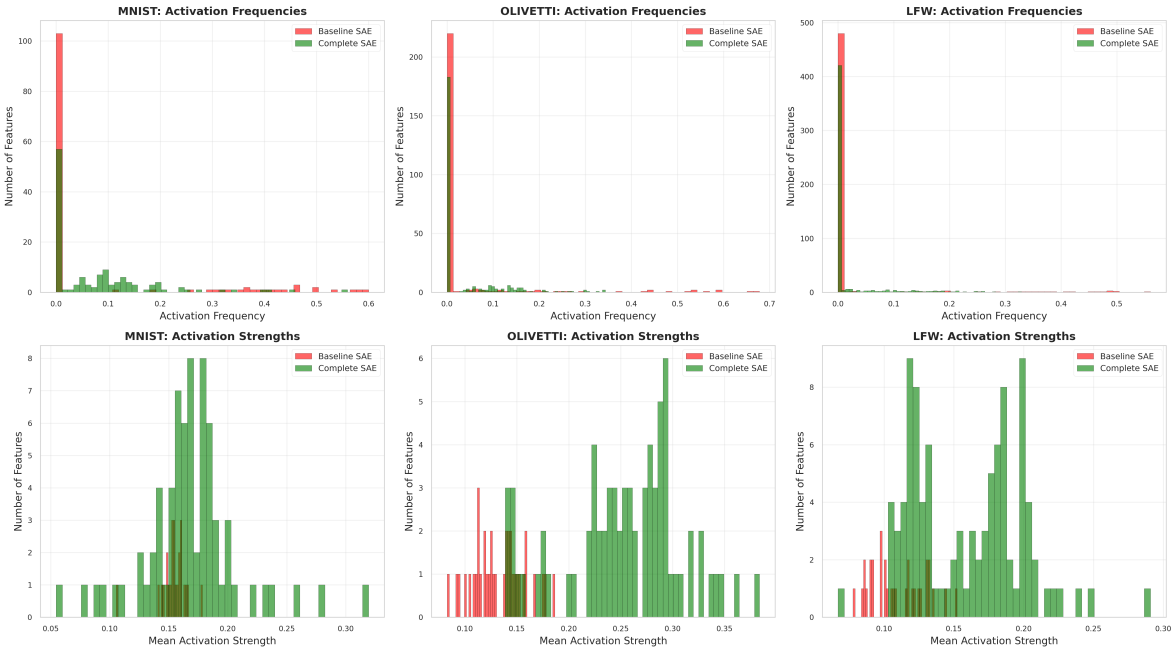

Figure 3: Activation frequency and strength distributions comparing baseline versus Complete SAE (combined techniques). **Top**: Frequency histograms show Complete SAE reduces near-zero features by 40-50% and eliminates winner-take-all spikes, achieving balanced 0.05-0.20 distribution. **Bottom**: Strength histograms reveal 3-4× improvements, with Complete SAE reaching 0.15-0.30 versus baseline's 0.05-0.10, indicating more confident, interpretable features [GDlTT+24b, OF96].

3.3.4 Discussion

Combined techniques yield synergistic effects: initialization prevents initial collapse while auxiliary loss strengthens resurrected features [GDlTT+24b]. Reducing dead neurons from 70% to 30-40% effectively doubles model capacity without computational cost, improving both efficiency and interpretability [GDlTT+24b]. The 3-4× activation strength increase enables cleaner additive decompositions where each feature contributes decisively: $x \approx \sum_{i \in \text{active}} h_i W_{\text{dec},i}$ with each $W_{\text{dec},i}$ representing an interpretable part [LS99]. These improvements align with the goal of discovering human-interpretable building blocks that decompose complex data into understandable components [GDlTT+24b, EHO+22].

3.3.5 Effect of K on Reconstruction Quality - and parts based representation

Hypothesis We hypothesize that reconstruction quality exhibits an inverse relationship with sparsity level k, as fewer active neurons constrain representational capacity and increase reconstruction difficulty [MF13]. However, we expect this relationship to plateau at higher k values where sparsity provides sufficient capacity for accurate reconstruction [GDlTT+24b].

Experimental Setup We systematically vary $k \in \{1, 2, 3, 4, 5, 10, 15, ..., 60\}$ across MNIST, Olivetti Faces, and LFW with hidden dimensions $n \in \{128, 256\}$, training each configuration for 25 epochs [GDlTT+24b]. All models employ tied weight initialization and maintain consistent hyperparameters (batch size 128, $\eta = 10^{-3}$, Adam) [KB14]. We measure mean squared error (MSE) for reconstruction fidelity and track active feature counts to assess dictionary utilization [GDlTT+24b].

Results

results on MSE description and plot with proper mse

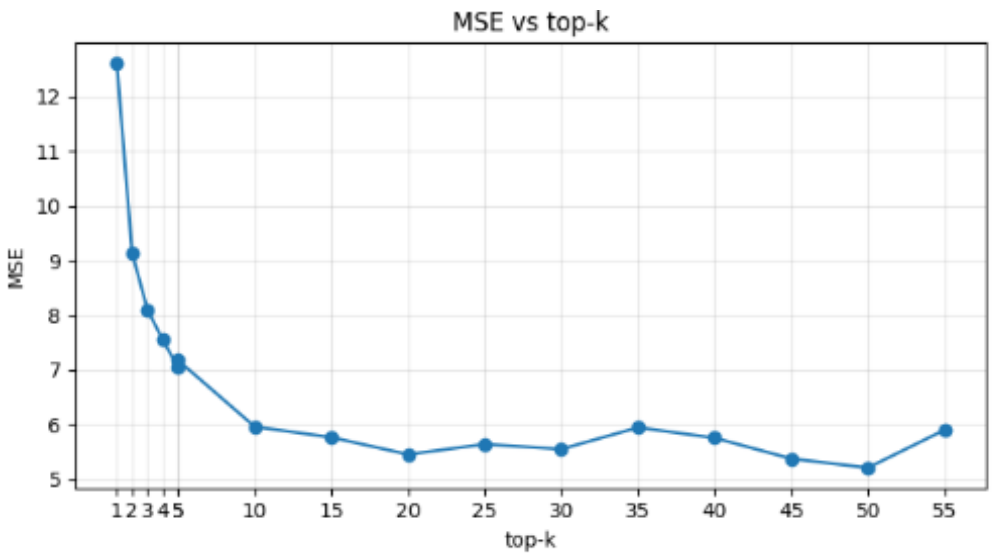

Figure 4: Top K versus MSE in increments of 5. 25 epochs per top k value

Figure 5 reveals a striking qualitative shift in learned feature representations as k increases [LS99]. At $k = 5$, decoder weights represent holistic, digit-level features where each learned component resembles complete handwritten digits (e.g., entire "3", "7", "8") [LS99]. With only 5 active features per reconstruction, the model is forced to encode whole-object templates rather than compositional parts [LS99]. As k increases to 32, features transition to intermediate-scale components such as digit segments, loops, and stroke combinations [LS99]. At $k = 64$, learned features become highly localized, representing fine-grained primitives including individual edges, curves, corners, and small stroke fragments [LS99, OF96]. This progression demonstrates that higher sparsity budgets enable genuine parts-based decomposition, where complex digits are reconstructed through additive composition of simple, interpretable building blocks [LS99].

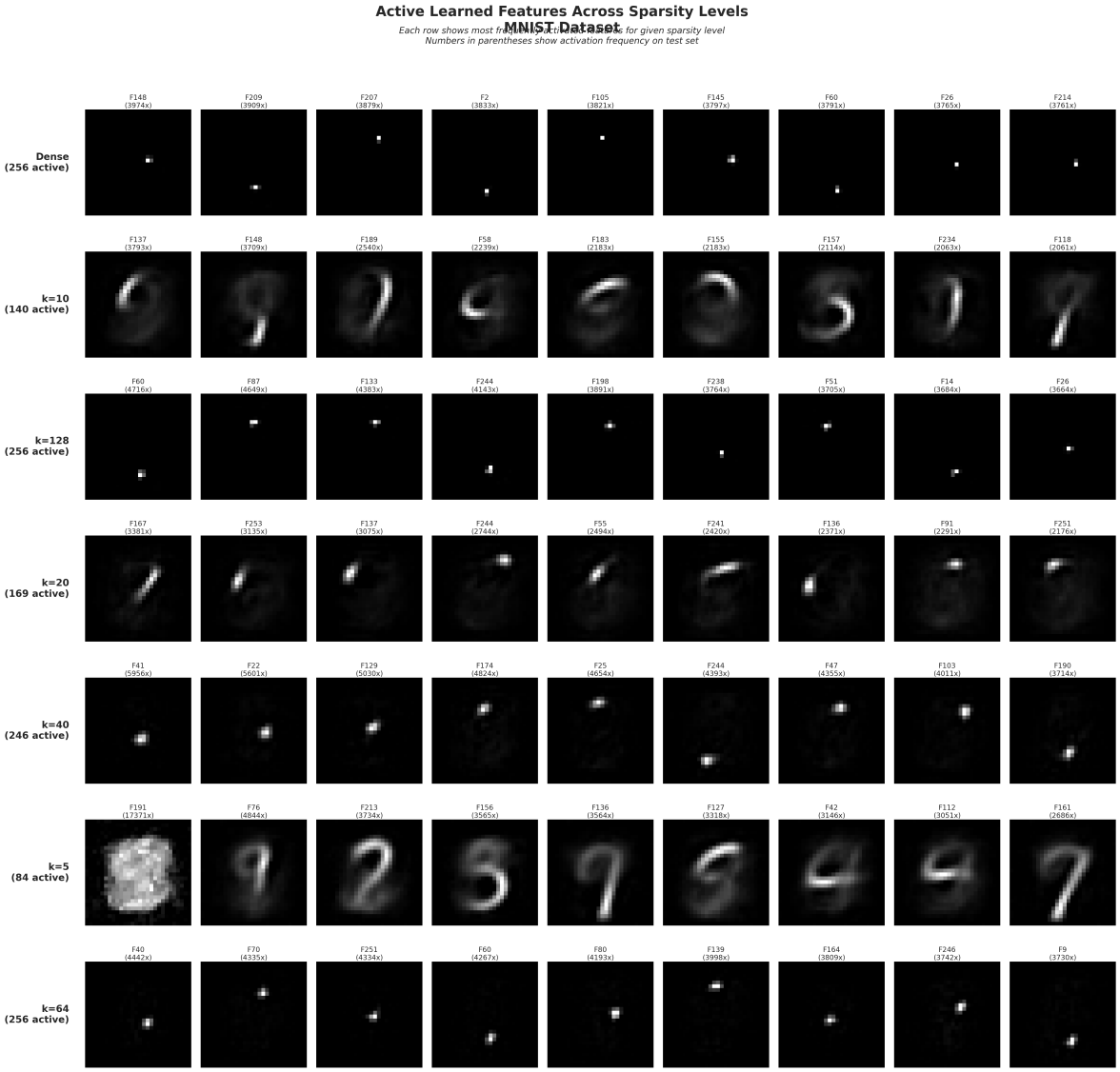

Figure 5: Visualization of learned decoder weights (most frequently activated features) from MNIST TopK SAE trained at different sparsity levels. At low sparsity ($k = 5$), features represent holistic, digit-level templates resembling complete characters. At intermediate sparsity ($k = 32$), features transition to digit segments and stroke combinations. At high sparsity ($k = 64$), features become fine-grained primitives (edges, curves, corners) enabling true parts-based additive decomposition. Increased sparsity budget shifts representation from template matching to compositional parts-based reconstruction [LS99, OF96].

organize by k

Discussion Experimental results demonstrate that NMSE exhibits an inverse relationship with k (Figure 4). More specifically, reconstruction error can be approximated by a $\frac{1}{k}$-type decay function. Notably, Figure 4 demonstrates convergence to a stable reconstruction error plateau rather than continued overfitting, suggesting the sparsity constraint provides effective regularization even at higher k values. This finding aligns with theoretical analysis showing that k-sparse autoencoders learn incoherent dictionaries that enable stable sparse recovery [MF13].

A critical advantage of TopK over L_1 regularization is the elimination of activation shrinkage—a known drawback where the L_1 penalty tends to shrink all activations toward zero [Tib96, GDlTT+24a]. TopK entirely removes the need for an L_1 penalty, preventing this shrinkage artifact [GDlTT+24a].

3.3.6 BatchTopK

A promising variant of Topk is BatchTopK which sets the average activations equal to k rather than a set constraint of k activations per sample. We aim to confirm previous emperical results [LBMP+25, BLN24] of BatchTopK achieving a lower MSE compared to TopK.

Experimental Setup Following *BatchTopK* [BLN24] we flatten all feature activations across the batch, and then take the top $\hat{k} = k \cdot \text{batch_size}$ activations where k is a value similar to normal TopK. Using this method we get the $\hat{k}$ largest activations over all samples in the batch, making the average activation per sample equal to exactly k.

Results

add results of these two here

Discussion BatchTopK, like TopK is hard sparsity constraint, meaning it does not affect the loss function.

3.4 Cross-Method Comparison

comparison to be done below

3.4.1 KL-Divergence vs TopK

- Hypothesis: TopK achieves better sparsity-fidelity tradeoff - Experimental setup: Matched sparsity levels - Results: MSE, dead neurons, training time - Pareto frontier plot

3.4.2 Kl-Divergence vs BatchTopK

3.4.3 Combined Techniques

- Hypothesis: Multiple techniques compound benefits - Test: TopK + AuxK + Tied Init vs baseline - Results, Discussion

3.5 Interpretability Assessment

3.5.1 Visual Inspection of Learned Features

- Compare decoder weights visualization across methods - Parts-based decomposition quality

3.5.2 Reconstruction Quality Analysis

- Qualitative examples - Quantitative metrics

3.6

4 Conclusions

References

[BLN24] Bart Bussmann, Patrick Leask, and Neel Nanda. Batchtopk: A simple improvement for topk-saes. AI Alignment Forum post, 2024. Accessed: 2025-10-21.

[BPY+21] Tolga Bolukbasi, Adam Pearce, Ann Yuan, Andy Coenen, Emily Reif, Fernanda Viégas, and Martin Wattenberg. An interpretability illusion for BERT. *arXiv preprint arXiv:2104.07143*, 2021.

[DDS+09] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255. IEEE, 2009.

[EHO+22] Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, Roger Grosse, Sam McCandlish, Jared Kaplan, Dario Amodei, Martin Wattenberg, and Christopher Olah. Toy models of superposition. *arXiv preprint arXiv:2209.10652*, 2022.

[GB10] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feed-forward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, pages 249–256. JMLR Workshop and Conference Proceedings, 2010.

[GBB11] Xavier Glorot, Antoine Bordes, and Yoshua Bengio. Deep sparse rectifier neural networks. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 315–323. JMLR Workshop and Conference Proceedings, 2011.

[GDlTT+24a] Leo Gao, Tom Dupré la Tour, Henk Tillman, Gabriel Goh, Rajan Tow, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. Scaling and evaluating sparse autoencoders. *OpenAI*, 2024.

[GDlTT+24b] Leo Gao, Tom Dupré la Tour, Henk Tillman, Gabriel Goh, Rajan Tow, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. Scaling and evaluating sparse autoencoders. *arXiv preprint arXiv:2406.04093*, 2024.

[KB14] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014. Published as conference paper at ICLR 2015.

[LBBH98] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[LBMP+25] Patric Leask, Bart Bussmann, Joseph Bloom Micheal Pearce, Curt Tigges, Noura Al Moubayed, and Lee Sharkey. Sparse autoencoders do not find canonical units of analysis. In *ICLR 2025*, 2025.

[LS99] Daniel D. Lee and H. Sebastian Seung. Learning the parts of objects by non-negative matrix factorization. *Nature*, 401(6755):788–791, 1999.

[MF13] Alireza Makhzani and Brendan Frey. k-sparse autoencoders. *arXiv preprint arXiv:1312.5663*, 2013. Available: https://arxiv.org/pdf/1312.5663.pdf.

[Ng11] Andrew Ng. Sparse autoencoder. *CS294A Lecture notes*, 72:1–19, 2011.

[OF96] Bruno A. Olshausen and David J. Field. Emergence of simple-cell receptive field properties by learning a sparse code for natural images. *Nature*, 381(6583):607–609, 1996.

[RLS+24] Senthooran Rajamanoharan, Tom Lieberum, Nicolas Sonnerat, Arthur Conmy, Vikrant Varma, János Kramár, and Neel Nanda. Jumping ahead: Improving reconstruction fidelity with jumprelu sparse autoencoders. *arXiv preprint arXiv:2407.14435*, 2024. Version 3.

[SH94] Ferdinando S Samaria and Andy C Harter. Parameterisation of a stochastic model for human face identification. In *Proceedings of 1994 IEEE Workshop on Applications of Computer Vision*, pages 138–142. IEEE, 1994.

[Tib96] Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1):267–288, 1996.

5 New Research

5.1 Metrics for Comparing NNSEA